OPEN BANKING API GATEWAY & DEVELOPER PORTAL DESIGN

Category: Open Banking | API Economy

Timeline: 7 Days | AI-Accelerated Architecture & Product Design Sprint

Target: Undergraduate / MBA / Engineering (CS/IT) / FinTech / Product & API Enthusiasts

TASK:

Design an Open Banking API Gateway and Developer Portal using industry standards such as OpenAPI 3.0, OAuth 2.0, and FAPI security protocols. Work with a simulated banking environment covering account information, payments, consent management, and event notifications across global frameworks like PSD2, UK Open Banking, India's Account Aggregator, and Australia's CDR. Develop API specifications, gateway architecture, consent lifecycle management, rate limiting strategies, and developer onboarding systems. Create sandbox environments, monitoring frameworks, and API governance policies including versioning and deprecation. Design a gamified "Open Gateway" simulation platform to analyse real-world API platform challenges through regulatory, technical, and developer experience lenses.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

This project document defines the comprehensive assessment framework for Project 1D under the Software Engineer Role at Zetheta Algorithms Private Limited. The project tasks the intern with architecting an Open Banking API Gateway and Developer Portal that is compliant with international open banking mandates including the European Union's Revised Payment Services Directive (PSD2), India's Account Aggregator framework, Australia's Consumer Data Right (CDR), and the United Kingdom's Open Banking Standard.

The intern assumes the role of API Platform Lead at a mid-size bank that has received a regulatory mandate to launch its Open Banking programme within a compressed timeline. The project is deliberately designed to simulate the competing pressures that define real-world platform engineering: the compliance team demands ironclad security, the business team demands speed-to-market, and external third-party developers demand usability and frictionless onboarding. The intern must navigate these tensions while producing production-grade technical artefacts.

Over seven days, the intern will design a complete API gateway architecture with security layer specifications, author 20 endpoint specifications in OpenAPI 3.0 format, design OAuth 2.0 and FAPI (Financial-grade API) security flows, create developer portal wireframes and documentation structures, formulate rate limiting and throttling strategies, specify sandbox environment architecture, and draft API versioning and deprecation policies.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

This section provides the foundational knowledge required to execute the project. The intern is expected to study these materials thoroughly before commencing design work. Each sub-section maps directly to one or more assessment competencies.

SECTION A1: OPEN BANKING FUNDAMENTALS

A1.1 What is Open Banking?

Open Banking is a regulatory and technological framework that mandates financial institutions to share customer financial data with authorised third-party providers (TPPs) through secure Application Programming Interfaces (APIs), subject to explicit customer consent. The movement represents a fundamental shift from proprietary, closed banking systems to interoperable, API-driven ecosystems that foster competition, innovation, and consumer choice.

The core principle is straightforward: a customer's financial data belongs to the customer, not the bank. If a customer consents to share their transaction history, account balances, or payment capabilities with a fintech application, the bank must facilitate that sharing through standardised, secure APIs. This principle is variously encoded in law across jurisdictions.

A1.2 PSD2 - European Union

The Revised Payment Services Directive (PSD2) is the European Union's legislative framework for open banking, effective since January 2018 with Strong Customer Authentication (SCA) enforcement from September 2019. PSD2 mandates that banks (known as Account Servicing Payment Service Providers, or ASPSPs) provide dedicated interfaces (APIs) for two categories of regulated third-party providers:

- **Account Information Service Providers (AISPs):** Authorised to access customer account information (balances, transaction history, standing orders) with customer consent. Use case: personal finance management apps, creditworthiness assessment, accounting software integration.
- **Payment Initiation Service Providers (PISPs):** Authorised to initiate payments directly from a customer's bank account. Use case: merchant checkout flows that bypass card networks, bill payment services, peer-to-peer transfers.

PSD2 requires banks to implement a "dedicated interface" (i.e., a purpose-built API) or allow TPPs to access through the customer-facing interface (screen scraping fallback) if the dedicated interface is unavailable or underperforming. The Berlin Group, STET (France), and Open Banking UK have published competing technical standards for PSD2 API implementation.

Regulatory Technical Standards (RTS) on SCA: The RTS specify that Strong Customer Authentication must combine at least two of three factors: something the customer knows (PIN, password), something the customer has (phone, hardware token), and something the customer is

(biometric). SCA is required for all electronic payments and account access, with defined exemptions for low-value transactions, trusted beneficiaries, recurring payments, and merchant-initiated transactions.

A1.3 UK Open Banking Standard

The UK's Competition and Markets Authority (CMA) ordered the nine largest UK banks to adopt open banking standards in 2017, predating PSD2 enforcement. The Open Banking Implementation Entity (OBIE) developed detailed API specifications, a trust framework (the Open Banking Directory), and conformance testing tools. Key characteristics that distinguish the UK model:

- Read/Write Data API specifications versioned at v3.1.x, providing granular endpoint definitions for accounts, balances, transactions, beneficiaries, standing orders, direct debits, products, offers, parties, and statements.
- Payment Initiation APIs supporting domestic payments, domestic scheduled payments, domestic standing orders, international payments, and file payments (bulk).
- Event Notification APIs enabling real-time webhooks for consent revocation, account changes, and payment status updates.
- A centralised Directory for TPP registration, certificate issuance (OBWAC/OBSEAL), and software statement assertions.
- Mandatory Customer Experience Guidelines (CEG) ensuring banks provide consistent, non-discriminatory consent journeys for TPP customers.

A1.4 India Account Aggregator (AA) Framework

India's Account Aggregator framework, operationalised by the Reserve Bank of India (RBI) and the Sahamati collective, takes a consent-mediated data-sharing approach distinct from both PSD2 and UK Open Banking. The architecture defines four key participants:

- **Financial Information User (FIU):** The entity requesting data (e.g., a lending institution requesting bank statements for credit assessment).
- **Financial Information Provider (FIP):** The entity holding the data (e.g., the customer's bank).
- **Account Aggregator (AA):** A licensed consent manager that facilitates data flow between FIU and FIP. The AA never stores or processes the financial data itself - it is a blind pipe.
- **Customer:** The individual whose data is being shared, who must provide explicit, granular consent.

The AA framework uses a consent artefact (a digitally signed JSON object) that specifies: purpose of data access, data categories (deposit, investment, insurance, tax), date range of data requested, frequency of access (one-time, periodic), and data life (how long the FIU may retain the data after

receipt). This granular consent model is considered more privacy-preserving than PSD2's binary consent approach.

A1.5 Australia Consumer Data Right (CDR)

Australia's Consumer Data Right is a cross-sector data portability framework that began with banking (Open Banking) and is extending to energy and telecommunications. Administered by the Australian Competition and Consumer Commission (ACCC), the CDR mandates that accredited data recipients can access consumer data through standardised APIs. The CDR uses a tiered accreditation model: unrestricted accreditation (full API access, stringent security requirements), sponsored accreditation (leveraging a sponsor's accreditation), and CDR representative (operating under an accredited entity). The technical standards are published by the Data Standards Body and mandate FAPI 1.0 Advanced as the security profile.

A1.6 Comparative Analysis

Dimension	PSD2 (EU)	UK Open Banking	India AA	Australia CDR
Regulator	European Banking Authority	CMA / FCA	RBI / Sahamati	ACCC / OAIC
Scope	Payment accounts	CMA9 banks + voluntary	All financial data	Banking + energy + telco
Security Profile	eIDAS certificates	OBWAC/OBSEAL + FAPI	PKI + consent artefact	FAPI 1.0 Advanced
Consent Model	Binary (yes/no per TPP)	Granular per data cluster	Purpose-bound artefact	Granular + tiered access
Data Intermediary	None (direct API)	None (direct API)	Account Aggregator (blind pipe)	None (direct API)
API Standard	Berlin Group / STET / OBIE	OBIE v3.1.x	ReBIT AA API v2.0	CDS API v1.x
Payment Initiation	Yes (PISP)	Yes (v3.1.x)	No (read-only)	No (read-only, planned)

SECTION A2: API GATEWAY ARCHITECTURE

A2.1 What is an API Gateway?

An API gateway is a server that acts as the single entry point for all API requests from external consumers. It sits between the client (TPP application, developer's code) and the backend microservices that implement business logic. The gateway is responsible for request routing, composition, protocol translation, authentication, authorisation, rate limiting, request/response transformation, caching, logging, monitoring, and circuit breaking.

In the context of open banking, the API gateway is the critical infrastructure component that stands between the outside world (thousands of TPP applications) and the bank's core systems (account ledgers, payment engines, customer databases). Every request and response passes through the gateway, making it the enforcement point for security policies, regulatory compliance, SLA management, and data governance.

A2.2 Gateway Architecture Patterns

Pattern 1 - Centralised Gateway: A single gateway instance (or cluster) handles all API traffic. This is the simplest pattern and suitable for smaller banks with limited API surface area. Pros: single point of configuration, unified logging, simple certificate management. Cons: single point of failure, monolithic configuration, blast radius of misconfiguration affects all APIs.

Pattern 2 - Federated Gateway (Gateway per Domain): Multiple gateway instances, each dedicated to a specific API domain (accounts, payments, consent, onboarding). Each gateway is independently configured, scaled, and deployed. Pros: blast radius containment, independent scaling (payments may need 10x the capacity of accounts), domain-specific security policies. Cons: operational complexity, cross-cutting concern duplication.

Pattern 3 - BFF (Backend-for-Frontend) + Gateway: A lightweight BFF layer sits behind the main gateway and provides client-specific response shaping. The BFF aggregates calls to multiple backend services and returns a tailored payload. Pros: optimised payloads per client type (mobile SDK vs. server-to-server), reduced chattiness. Cons: additional latency hop, BFF proliferation.

Pattern 4 - Service Mesh + Thin Gateway: A service mesh (Istio, Linkerd) handles inter-service communication, mTLS, observability, and traffic management internally, while a thin gateway handles only external-facing concerns (public TLS termination, developer authentication, rate limiting). Pros: separation of external vs. internal concerns, reduced gateway complexity. Cons: requires Kubernetes maturity, operational learning curve.

A2.3 Key Gateway Components for Open Banking

- **TLS Termination Layer:** Terminates TLS 1.2/1.3 connections from TPPs. In PSD2 environments, this layer validates eIDAS QWAC certificates presented by TPPs. In UK Open Banking, it validates OBWAC transport certificates issued by the Open Banking Directory.

- **Authentication & Authorisation Engine:** Validates OAuth 2.0 access tokens (introspection or local JWT validation), checks scopes against the requested endpoint, enforces consent boundaries (e.g., the token grants access to account 12345 but the request is for account 67890 - reject).
- **Request Validation:** Validates incoming requests against the published OpenAPI 3.0 schema. Rejects malformed payloads, missing required headers (x-fapi-interaction-id, x-fapi-financial-id), and unsupported content types before the request reaches backend services.
- **Rate Limiter:** Enforces per-TPP, per-endpoint, and global rate limits. Open banking regulations typically mandate minimum performance thresholds (e.g., UK OBIE mandates 99.5% availability and response times under 1.5 seconds for 95th percentile).
- **Response Transformer:** Transforms internal service responses into the standardised open banking response format. Strips internal fields, adds mandatory response headers (x-fapi-interaction-id echo, content-type), and handles pagination metadata.
- **Audit Logger:** Logs every request and response with sufficient detail for regulatory audit (timestamp, TPP identifier, endpoint, consent ID, response code, latency) without logging sensitive customer data (account numbers, balances).
- **Circuit Breaker:** Protects backend core banking systems from cascading failures. If the accounts service is degraded, the circuit breaker returns a standardised error response rather than allowing requests to queue and timeout.

A2.4 Technology Landscape

Gateway	Type	Strengths for Open Banking	Considerations
Kong Gateway	Open Source / Enterprise	Plugin ecosystem, declarative config, Kubernetes-native, DB-less mode	Plugin development in Lua/Go, enterprise features paywalled
Apigee (Google)	Managed	API analytics, developer portal built-in, monetisation, traffic management	Cost at scale, vendor lock-in, learning curve
AWS API Gateway	Managed	Lambda integration, AWS ecosystem, WebSocket support, regional deployment	Cold start latency, limited customisation, AWS-only
Tyk Gateway	Open Source / Enterprise	Go-based (performant), GraphQL support, UDG (Universal Data Graph)	Smaller community, fewer pre-built plugins

Gateway	Type	Strengths for Open Banking	Considerations
WSO2 API Manager	Open Source	Full lifecycle management, identity server integration, analytics	Heavy Java stack, complex deployment
Gravitee.io	Open Source	Event-native, policy studio, API designer, developer portal	Younger ecosystem, enterprise support tiers

SECTION A3: OAUTH 2.0 & FAPI SECURITY ARCHITECTURE

A3.1 OAuth 2.0 Fundamentals for Open Banking

OAuth 2.0 is the authorisation framework that underpins all open banking API access. In the open banking context, OAuth 2.0 enables a customer to grant a third-party provider (TPP) limited, revocable access to their financial data without sharing their banking credentials. The four primary roles in the OAuth 2.0 open banking flow are:

- **Resource Owner:** The bank customer who owns the financial data.
- **Client:** The TPP application requesting access to the customer's data.
- **Authorisation Server:** The bank's OAuth server that authenticates the customer, collects consent, and issues access/refresh tokens.
- **Resource Server:** The bank's API gateway/backend that serves the protected resources (account data, payment endpoints).

A3.2 Authorization Code Flow with PKCE

The Authorization Code Flow is the only grant type permitted for open banking user-delegated access. The Proof Key for Code Exchange (PKCE) extension is mandatory to prevent authorization code interception attacks. The flow proceeds as follows:

- **TPP Registration:** The TPP registers with the bank's developer portal, obtains a client_id, and registers redirect_uris. For FAPI compliance, the TPP also provides its transport certificate (mTLS) and signing certificate.
- **Consent Creation:** The TPP calls the bank's Consent API to create a consent resource specifying the permissions requested (ReadAccountsDetail, ReadBalances, ReadTransactionsDetail, etc.), the expiration date, and the transaction date range.
- **PKCE Challenge Generation:** The TPP generates a high-entropy code_verifier (43–128 characters, URL-safe), computes the code_challenge as BASE64URL(SHA256(code_verifier)), and includes both code_challenge and code_challenge_method=S256 in the authorization request.
- **Authorization Request:** The TPP redirects the customer's browser to the bank's authorization endpoint with parameters: response_type=code, client_id, redirect_uri, scope, state (CSRF protection), nonce (replay protection), code_challenge, code_challenge_method, and a request object (signed JWT containing all parameters for FAPI compliance).
- **Customer Authentication & Consent:** The bank authenticates the customer (SCA: password + OTP/biometric), displays the consent screen showing which data the TPP is requesting, and collects explicit consent.

- **Authorization Code Issuance:** Upon consent, the bank redirects back to the TPP's redirect_uri with an authorization code and the original state parameter. The code is short-lived (60–90 seconds) and single-use.
- **Token Exchange:** The TPP exchanges the authorization code for an access token and refresh token by calling the token endpoint. The TPP includes the code_verifier - the bank verifies that SHA256(code_verifier) matches the original code_challenge. For FAPI, this call is authenticated via mTLS client certificate.
- **API Access:** The TPP calls the resource server (API gateway) with the access token in the Authorization: Bearer header. The gateway validates the token, checks scopes and consent boundaries, and proxies the request to the backend.

A3.3 FAPI 1.0 Advanced Profile

The Financial-grade API (FAPI) security profile, developed by the OpenID Foundation, hardens OAuth 2.0 for high-security financial APIs. FAPI 1.0 Advanced is mandated by Australia CDR, recommended by UK Open Banking v3.1.x, and increasingly adopted in other jurisdictions. Key requirements beyond baseline OAuth 2.0:

- **Mutual TLS (mTLS):** TPPs must present an X.509 client certificate during TLS handshake. The certificate is bound to the TPP's client_id at registration. Access tokens are certificate-bound (sender-constrained) - a stolen token cannot be used without the corresponding private key.
- **Signed Request Objects (JAR):** Authorization requests must be passed as signed JWTs (request parameter or request_uri), preventing parameter tampering in the browser redirect.
- **Signed & Encrypted ID Tokens:** ID tokens returned by the authorization server must be signed (RS256/PS256/ES256) and optionally encrypted (A256GCM). The ID token must contain the s_hash (state hash), c_hash (code hash), and at_hash (access token hash) claims.
- **PAR (Pushed Authorization Requests):** The authorization request is first pushed to a dedicated PAR endpoint (backchannel, not browser), which returns a request_uri. The browser redirect then references this request_uri instead of carrying parameters in the URL. This prevents request leakage via browser history, referrer headers, and proxy logs.
- **JARM (JWT Secured Authorization Response Mode):** The authorization response (code + state) is returned as a signed JWT rather than plain query parameters, preventing response tampering.

A3.4 Consent Management Architecture

Consent is the cornerstone of open banking. The consent lifecycle must be precisely managed:

Consent Creation: The TPP creates a consent resource specifying requested permissions, validity period, and data range. The consent is in 'AwaitingAuthorisation' status.

Consent Authorisation: The customer authenticates and approves the consent. Status moves to 'Authorised'. An authorization code is issued.

Consent Active: Tokens are issued against the consent. The TPP can make API calls within the consent boundaries. The bank must track which data has been accessed under which consent.

Consent Revocation: The customer can revoke consent at any time via the bank's interface. The bank must immediately invalidate all tokens issued under that consent and notify the TPP via the Event Notification API (webhook or polling).

Consent Expiry: Consents have a defined expiration. Upon expiry, tokens are invalidated and the TPP must request fresh consent. UK Open Banking limits long-lived consents to 90 days for AIS.

SECTION A4: OPENAPI 3.0 SPECIFICATION DESIGN

A4.1 OpenAPI 3.0 Structure

The OpenAPI Specification (OAS) 3.0 is the industry standard for describing RESTful APIs. For open banking, the OAS definition serves as the single source of truth for API behaviour. It defines endpoints, request/response schemas, authentication methods, error formats, and documentation metadata. A well-crafted OAS definition enables automated code generation (server stubs, client SDKs), contract testing, documentation generation, and mock server creation.

The key sections of an OpenAPI 3.0 document for open banking:

- **info:** API title, version, description, contact, license, and terms of service URL.
- **servers:** Base URLs for production, sandbox, and staging environments.
- **paths:** Endpoint definitions with HTTP methods, parameters, request bodies, responses, security requirements, and operation IDs.
- **components/schemas:** Reusable data models (Account, Balance, Transaction, Consent, Payment, etc.) with JSON Schema validation rules.
- **components/securitySchemes:** OAuth 2.0 configuration (authorization URL, token URL, scopes), mTLS specification, and API key definitions.
- **components/parameters:** Reusable path and query parameters (x-fapi-interaction-id, x-fapi-financial-id, pagination parameters).
- **components/responses:** Reusable error response definitions (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests, 500 Internal Server Error).

A4.2 Mandatory Endpoints (20 Specifications)

The intern must design OpenAPI 3.0 specifications for a minimum of 20 endpoints across four API domains:

#	Endpoint	Method	Domain	Description
1	/consents	POST	Consent	Create a new consent resource with requested permissions and validity period
2	/consents/{consentId}	GET	Consent	Retrieve consent status and details
3	/consents/{consentId}	DELETE	Consent	Revoke an existing consent and invalidate all associated tokens

#	Endpoint	Method	Domain	Description
4	/accounts	GET	Accounts	List all accounts the TPP has consent to access
5	/accounts/{accountId}	GET	Accounts	Retrieve detailed account information (type, currency, status, identifiers)
6	/accounts/{accountId}/balances	GET	Accounts	Retrieve current and available balances for an account
7	/accounts/{accountId}/transactions	GET	Accounts	Retrieve paginated transaction history with date range filtering
8	/accounts/{accountId}/beneficiaries	GET	Accounts	List registered beneficiaries for an account
9	/accounts/{accountId}/standing-orders	GET	Accounts	List standing orders (recurring payments) for an account
10	/accounts/{accountId}/direct-debits	GET	Accounts	List direct debit mandates for an account
11	/domestic-payments	POST	Payments	Initiate a domestic payment (requires payment consent)
12	/domestic-payments/{paymentId}	GET	Payments	Retrieve payment status and details
13	/domestic-payment-consents	POST	Payments	Create payment-specific consent resource
14	/international-payment-consents	POST	Payments	Create consent resource for international payment initiation
15	/international-payment-consents/{consentId}	GET	Payments	Retrieve international payment consent status
16	/international-payments	POST	Payments	Initiate an international payment with currency conversion
17	/international-payments/{paymentId}	GET	Payments	Retrieve international payment status and details
18	/event-subscriptions	POST	Events	Subscribe to event notifications (consent revocation, payment status)
19	/event-subscriptions/{subId}	GET	Events	Retrieve event subscription details
20	/events	POST	Events	Receive aggregated event notifications (polling model)

SECTION A5: DEVELOPER PORTAL & EXPERIENCE DESIGN

A5.1 Why Developer Experience (DX) Matters in Open Banking

The success of an Open Banking programme is not determined solely by the technical quality of the APIs - it is determined by how quickly and successfully external developers can integrate with those APIs. If a TPP developer cannot understand, authenticate, test, and go live with your APIs within a reasonable timeframe, they will choose a competitor bank's APIs. Developer Experience (DX) is therefore a first-class product concern, not an afterthought.

Research from Stripe, Twilio, and Plaid consistently shows that developer portal quality is the single largest predictor of API adoption. Key DX metrics include: Time to First API Call (TTFAC), Time to First Successful Integration, documentation Net Promoter Score (NPS), support ticket volume per integration, and sandbox-to-production conversion rate.

A5.2 Developer Portal Architecture

A production-grade developer portal for open banking must include the following components:

- **Registration & Onboarding:** Self-service developer registration, organisation creation, software statement assertion management, certificate upload (mTLS), and redirect URI configuration. The onboarding flow should be completable in under 15 minutes.
- **Interactive API Documentation:** Auto-generated from OpenAPI 3.0 specs with a 'Try It' feature that allows developers to make API calls directly from the documentation page against the sandbox environment. Tools: Redoc, Swagger UI, Stoplight Elements, ReadMe.
- **Getting Started Guide:** A step-by-step quickstart that takes a developer from zero to their first successful API call in under 30 minutes. Includes code samples in Python, Node.js, Java, and C#.
- **Authentication Guide:** Dedicated documentation for the OAuth 2.0 / FAPI flow with sequence diagrams, code examples, token management best practices, and common pitfalls.
- **Sandbox Environment:** A fully functional test environment with synthetic data (test accounts, test customers, test payment rails) that mirrors production API behaviour. The sandbox must support the full consent journey, not just static mock responses.
- **API Changelog:** Versioned changelog documenting every API change with impact assessment, migration guides, and deprecation timelines.
- **Status Page:** Real-time API health dashboard showing availability, latency percentiles, error rates, and scheduled maintenance windows.
- **Support Channels:** Developer forum, FAQ, ticketing system, and office hours. Slack/Discord community for informal support.

A5.3 Error Taxonomy for Open Banking APIs

A consistent, informative error taxonomy is one of the most impactful DX investments. Open banking APIs must return structured error responses that enable programmatic handling. The recommended error response structure:

```
{
  "Code": "OBEX.Field.Missing",
  "Id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "Message": "Required field [DebtorAccount.Identification] is
missing",
  "Path": "Data.Initiation.DebtorAccount.Identification",
  "Url": "https://docs.bank.com/errors/OBEX.Field.Missing"
}
```

Each error code should link to a documentation page with: what caused the error, how to fix it, and example corrected requests. This reduces support ticket volume dramatically.

SECTION A6: RATE LIMITING, THROTTLING & API GOVERNANCE

A6.1 Rate Limiting Strategies

Rate limiting protects backend systems from abuse and ensures fair resource allocation across TPPs. Open banking introduces regulatory constraints on rate limiting - banks cannot use rate limits to deliberately degrade TPP access (which would constitute 'obstacle creation' under PSD2 Article 32(3)). The rate limiting strategy must balance three objectives: backend protection, fair access, and regulatory compliance.

Token Bucket Algorithm: Each TPP is allocated a bucket of tokens that refills at a defined rate. Each API call consumes one token. When the bucket is empty, requests are throttled (HTTP 429). Pros: allows bursting (the bucket can accumulate tokens during quiet periods), simple to implement. Cons: doesn't differentiate between endpoint types.

Sliding Window Algorithm: Counts requests in a sliding time window (e.g., last 60 seconds). More predictable than token bucket because it doesn't allow arbitrary bursts. Cons: requires more state storage.

Tiered Rate Limits: Different limits for different endpoints based on backend cost. Accounts/balances endpoints (read-only, cacheable) get higher limits than payment initiation endpoints (write, expensive). Example: 300 req/min for accounts, 60 req/min for payments, 10 req/min for bulk operations.

A6.2 Response Headers for Rate Limiting

Every response must include rate limit headers so TPPs can implement client-side throttling:

```
X-RateLimit-Limit: 300
X-RateLimit-Remaining: 247
X-RateLimit-Reset: 1679558400
Retry-After: 30
```

A6.3 SLA and Performance Obligations

UK Open Banking mandates the following performance thresholds:

Metric	Threshold	Measurement
Availability	>= 99.5%	Per calendar quarter, excluding planned downtime
Response Time (P95)	<= 1.5 seconds	95th percentile, measured at gateway egress
Error Rate	<= 0.5%	5xx responses as percentage of total requests
Planned Downtime	<= 4 hours/quarter	Must be communicated 48 hours in advance

SECTION A7: API VERSIONING & DEPRECATION STRATEGIES

A7.1 Versioning Approaches

URI Versioning (/v1/, /v2/): The API version is embedded in the URL path. Pros: explicit, easy to route at the gateway level, clear in logs. Cons: breaks URI stability, cache invalidation across versions.

Header Versioning (Accept: application/vnd.bank.v2+json): The version is specified in the Accept header using a custom media type. Pros: URI stability, content negotiation support. Cons: harder to test in a browser, less visible in logs.

Query Parameter Versioning (?version=2): The version is a query parameter. Pros: easy to add. Cons: breaks caching (different URL = different cache entry), feels ad hoc.

Recommendation for Open Banking: URI versioning (/v3.1/) is the de facto standard in open banking (used by UK OBIE, Berlin Group NextGenPSD2, and CDS Australia). It provides maximum clarity and is aligned with regulatory expectations for version-specific conformance testing.

A7.2 Deprecation Policy

A well-defined deprecation policy is essential for the open banking ecosystem because TPPs build integrations against specific API versions. Abrupt changes break production systems. The recommended deprecation lifecycle:

- **Sunset Header:** When a version is marked for deprecation, include a Sunset HTTP header in all responses: Sunset: Sat, 31 Dec 2025 23:59:59 GMT. This is machine-readable and allows TPPs to build automated alerts.
- **Deprecation Notice:** Minimum 6-month notice before removal. Published on the developer portal changelog, emailed to all registered developers, and announced via the status page.
- **Migration Guide:** A detailed document showing field-by-field mapping between the old and new version, code examples for migration, and a summary of breaking vs. non-breaking changes.
- **Dual Running:** Both old and new versions run simultaneously during the migration window. Traffic to the deprecated version is monitored - TPPs still using it are contacted directly.
- **Sunset Enforcement:** After the deprecation window, the old version returns 410 Gone with a body pointing to the migration guide and new version URL.

SECTION A8: SANDBOX ENVIRONMENT DESIGN

A8.1 Sandbox Requirements for Open Banking

PSD2 Article 30(5) mandates that banks provide a testing facility (sandbox) to TPPs at least three months before the go-live date of their production APIs. The sandbox must support the full API lifecycle including onboarding, authentication, consent, data access, and payment initiation. Static mock servers that return hardcoded responses do not satisfy this requirement.

A8.2 Sandbox Architecture

A production-grade sandbox for open banking consists of:

- **Synthetic Data Engine:** Generates realistic but fictional customer profiles, accounts, transactions, and balances. The data must cover edge cases: multi-currency accounts, joint accounts, accounts in different states (active, suspended, closed), transactions with various merchant category codes (MCCs), and standing orders with different frequencies.
- **Simulated Consent Journey:** A mock bank login and consent screen that TPPs can redirect to during testing. The sandbox must support both the 'happy path' (customer approves) and error paths (customer declines, session timeout, authentication failure).
- **Simulated Payment Rails:** Payment initiation in the sandbox should simulate the full payment lifecycle: accepted → pending → settled (or rejected). Configurable delay parameters allow TPPs to test timeout handling.
- **Rate Limit Simulation:** The sandbox should enforce the same rate limits as production so TPPs can test their retry logic and backoff strategies.
- **Error Injection:** A mechanism for TPPs to trigger specific error scenarios (401, 403, 429, 500, timeout) to test their error handling. This can be via special test account IDs or request headers.

A8.3 Sandbox vs. Production Parity

Aspect	Sandbox	Production
Data	Synthetic, resettable	Real customer data
Authentication	Simplified SCA (test credentials)	Full SCA (biometric, OTP)
Certificates	Test certificates (self-signed OK)	Production eIDAS/OBWAC/OBSEAL
Payment Settlement	Simulated (no real money moves)	Real-time settlement
Rate Limits	Same as production	Same as sandbox
API Behaviour	Identical response schemas	Identical response schemas
Onboarding	Self-service, instant approval	Regulatory checks, manual review

SECTION A9: API MONITORING, OBSERVABILITY & INCIDENT RESPONSE

A9.1 The Three Pillars of Observability

Production API platforms require observability across three pillars: metrics (quantitative measurements over time), logs (discrete event records), and traces (request journey across distributed components). In open banking, each pillar has regulatory implications.

Metrics: Time-series numerical data aggregated at regular intervals. Key metrics for an open banking gateway include: request rate (requests per second, broken down by TPP, endpoint, and HTTP method), latency distribution (P50, P95, P99 measured at gateway egress), error rate (4xx and 5xx responses as a percentage of total), token issuance rate (OAuth tokens issued per minute, indicating consent activity), consent conversion rate (consents authorised divided by consents created, indicating friction in the consent flow), and availability (uptime percentage per calendar quarter, measured against the 99.5% regulatory threshold).

Logs: Structured event records capturing individual API interactions. For regulatory audit, every log entry must include: timestamp (ISO 8601 with timezone), TPP identifier (client_id or software statement assertion ID), endpoint path and HTTP method, consent ID (linking the request to the governing consent), response status code, response latency (milliseconds), x-fapi-interaction-id (correlation ID), and request/response size (bytes). Critical rule: logs must NEVER contain sensitive customer data (account numbers, balances, transaction details) in plain text. Personally identifiable information must be tokenised or masked.

Traces: Distributed traces track a single request's journey through the gateway and backend services. A typical open banking request might traverse: TLS termination → certificate validation → token introspection → scope/consent check → rate limit check → request validation → backend service call → response transformation → audit logging. Each hop adds latency, and traces help identify bottlenecks. Tools: Jaeger, Zipkin, AWS X-Ray, Datadog APM.

A9.2 Alerting Strategy

Alerts must be actionable, not noisy. The recommended alerting hierarchy for an open banking gateway:

Severity	Condition	Response Time	Notification Channel	Example
P1 - Critical	API availability < 99%	< 15 minutes	PagerDuty + SMS + phone call	Core banking backend unreachable, all API calls failing
P2 - High	P95 latency > 3 seconds	< 30 minutes	PagerDuty + Slack	Database connection pool exhausted, responses slow
P3 - Medium	Error rate > 2% for a single endpoint	< 2 hours	Slack + email	Payment endpoint returning 500 for a specific currency
P4 - Low	Single TPP exceeding rate limits	< 24 hours	Email + Jira ticket	Fintech startup hammering sandbox without backoff

A9.3 Incident Response for Open Banking

Open banking regulations (particularly PSD2 Article 33 and UK OBIE Operational Guidelines) impose specific incident response obligations:

- **Major Incident Notification:** If API availability falls below the mandated threshold, the bank must notify the regulator within a defined period (typically 4 hours for major incidents). The notification must include: incident timeline, root cause (preliminary), customer impact assessment, and remediation plan.
- **TPP Communication:** All registered TPPs must be notified of planned downtime (minimum 48 hours advance notice) and unplanned outages (as soon as practically possible) via the status page and direct email.
- **Post-Incident Review:** A blameless post-incident review (PIR) document must be produced within 5 business days. The PIR covers: timeline, root cause analysis (5 Whys), impact quantification (affected TPPs, failed requests, customer impact), corrective actions with deadlines, and preventive measures.
- **Quarterly Reporting:** Banks must publish quarterly API performance statistics (availability, response times, error rates) on their developer portal. UK OBIE publishes league tables comparing banks' API performance.

SECTION A10: DATA MODELS & SCHEMA DESIGN PATTERNS

A10.1 Account Resource Model

The Account resource is the foundational data model in any open banking API. A well-designed Account schema must support multiple account types (current/checking, savings, credit card, mortgage, loan), multiple currencies, multiple identifiers (sort code + account number, IBAN, BBAN, PAN for cards), and multiple status states (enabled, disabled, pending, deleted).

Key design decisions the intern must make:

- **Identification Strategy:** Should account IDs be exposed as the bank's internal identifiers or as opaque, externally-generated UUIDs? Best practice: always use opaque UUIDs in the API layer. Internal account numbers should never be exposed to TPPs as they enable social engineering and account enumeration attacks.
- **Nested vs. Flat:** Should the Account response embed balances and transactions (nested) or require separate API calls (flat)? Best practice: flat with links. Nested responses create over-fetching, complicate caching, and break consent granularity (a TPP might have consent for account details but not balances).
- **Envelope Pattern:** All responses should use a consistent envelope: { Data: { ... }, Links: { Self, First, Prev, Next, Last }, Meta: { TotalPages, FirstAvailableDateTime, LastAvailableDateTime } }. The envelope separates business data from pagination metadata and HATEOAS links.

A10.2 Transaction Resource Model

Transactions are the most frequently accessed and most voluminous data type in open banking. Design considerations:

- **Pagination:** Cursor-based pagination (using a transaction ID or timestamp as the cursor) is strongly preferred over offset-based pagination for transactions. Offset pagination becomes increasingly slow as the offset grows (the database must scan and skip rows). Cursor pagination is O(1) regardless of position in the dataset.
- **Date Filtering:** Mandatory query parameters: fromBookingDateTime and toBookingDateTime (ISO 8601). The maximum date range should be configurable (90 days is typical). Requests without date parameters should default to the last 90 days, not all history.
- **Status:** Transactions have two key statuses: Booked (settled, final) and Pending (authorised but not settled). Some jurisdictions require separate endpoints or query parameters for pending vs. booked transactions.
- **Merchant Information:** Transaction records should include: merchant name, merchant category code (MCC), and optionally structured merchant address. This data is critical for TPP use cases like personal finance management and expense categorisation.

A10.3 Payment Resource Model

Payment initiation resources require careful schema design because they involve real money movement. Key patterns:

- **Idempotency:** Every POST /domestic-payments request must include an x-idempotency-key header. The bank must store this key and, for duplicate requests within a defined window (24-48 hours), return the original response without re-executing the payment. This prevents double payments caused by network retries, client bugs, or gateway timeouts.
- **Risk Assessment:** Payment requests must include a Risk object containing: PaymentContextCode (BillPayment, EcommerceGoods, EcommerceServices, PersonToPerson, Other), MerchantCategoryCode, MerchantCustomerIdentification, and DeliveryAddress. This data feeds into the bank's fraud detection engine.
- **Payment Status Lifecycle:** AcceptedSettlementInProcess → AcceptedSettlementCompleted (success path). AcceptedSettlementInProcess → Rejected (failure path). Each status transition should generate an event for the Event Notification API.

A10.4 Consent Resource Model - Detailed Schema

The consent resource is the most complex data model because it governs all subsequent API access. Minimum required fields:

Field	Type	Required	Description
ConsentId	string (UUID)	Generated	Unique identifier assigned by the bank upon creation
Status	enum	Generated	AwaitingAuthorisation | Authorised | Rejected | Revoked | Expired
StatusUpdateDateTime	datetime (ISO 8601)	Generated	Timestamp of last status change
CreationDateTime	datetime (ISO 8601)	Generated	When the consent was created
Permissions	array of strings	Required	ReadAccountsBasic, ReadAccountsDetail, ReadBalances, ReadTransactionsBasic, ReadTransactionsDetail, ReadBeneficiaries, ReadStandingOrders, ReadDirectDebits, ReadProducts, ReadOffers
ExpirationDateTime	datetime (ISO 8601)	Optional	When the consent expires (max 90 days for AIS in UK OBIE)
TransactionFromDateTime	datetime (ISO 8601)	Optional	Earliest transaction date the TPP can access
TransactionToDateTime	datetime (ISO 8601)	Optional	Latest transaction date the TPP can access

SECTION A11: COMMON ANTI-PATTERNS & PITFALLS IN OPEN BANKING API DESIGN

This section catalogues the most common mistakes made by banks implementing open banking APIs. The intern should use this as a checklist against their own design.

Anti-Pattern 1: Screen-Scraping-as-API

Some banks implement their open banking API by wrapping their internet banking HTML screens in an API layer - essentially performing server-side screen scraping of their own systems. This approach is fragile (breaks when the UI changes), slow (HTML parsing overhead), incomplete (UI often shows a subset of available data), and fundamentally misaligned with the API-first principle. The correct approach is to expose the same backend service layer that feeds the internet banking UI.

Anti-Pattern 2: Monolithic OpenAPI Specification

Some banks publish a single, massive OpenAPI specification file containing all endpoints (accounts, payments, events, consent) in one document. This creates: unwieldy documentation (developers must scroll through 5,000+ lines to find one endpoint), slow rendering in Swagger UI, and difficult versioning (a change to one endpoint triggers a version bump for the entire specification). Best practice: split specifications by domain (openapi-accounts.yaml, openapi-payments.yaml, openapi-consents.yaml, openapi-events.yaml).

Anti-Pattern 3: Consent-as-Afterthought

Banks that treat consent as a simple boolean flag (‘user said yes’) rather than a first-class resource with its own lifecycle will fail regulatory review. Consent must track: which permissions were granted, when consent was given, when it expires, which accounts are linked, and provide an audit trail of every data access made under the consent. The consent service must be independently scalable and independently auditable.

Anti-Pattern 4: Rate Limiting as Obstacle

PSD2 Article 32(3) explicitly prohibits banks from creating ‘obstacles’ to TPP access. Overly aggressive rate limiting (e.g., 10 requests per minute for accounts data) can be interpreted as an obstacle. The rate limiting policy must be justified by genuine technical necessity (backend protection), applied equally to the bank’s own channels and TPPs, and published transparently on the developer portal.

Anti-Pattern 5: Static Mock Sandbox

A sandbox that returns hardcoded JSON responses for every request (regardless of query parameters, consent state, or authentication) is worse than no sandbox. TPPs cannot test their consent flow, error

handling, or pagination logic against static mocks. The sandbox must be dynamic: it must enforce authentication, validate consent, return different data for different test accounts, and simulate the payment status lifecycle.

Anti-Pattern 6: Ignoring the Sunset Header

Banks that deprecate API versions without the Sunset HTTP header force TPPs to rely on email notifications and changelog monitoring. The Sunset header (RFC 8594) is machine-readable and allows automated deprecation alerting. Every response from a deprecated version must include: Sunset: [date], Deprecation: [date], and Link: <new-version-url>; rel="successor-version".

Anti-Pattern 7: Leaking Internal Identifiers

Exposing internal database IDs, core banking system account numbers, or internal service names in API responses creates security risks (enumeration attacks, internal topology disclosure) and tight coupling (changing the internal system breaks the external API). All external identifiers should be opaque UUIDs generated at the API layer.

Anti-Pattern 8: Missing Idempotency on Payment Endpoints

Payment initiation endpoints without idempotency keys are a critical defect. Network timeouts, client retries, and gateway failures can cause duplicate payment submissions. The x-idempotency-key header and server-side deduplication logic are non-negotiable for any endpoint that modifies state.

Anti-Pattern 9: Inconsistent Error Responses

Banks that return different error response structures for different endpoints (JSON for accounts, XML for payments, plain text for authentication errors) make it impossible for TPPs to build consistent error handling. The error response schema must be uniform across all endpoints: { Code, Id, Message, Path, Url } with every error code deep-linked to a documentation page.

Anti-Pattern 10: No API Changelog

Banks that make changes to their APIs without a public, versioned changelog force TPPs to discover breaking changes in production. Every API change - including non-breaking additions - must be documented in the changelog with: version number, date, type (breaking/non-breaking), affected endpoints, migration guide (if breaking), and deprecation timeline (if applicable).

PART B: GAMIFIED SIMULATION - "OPEN GATEWAY: THE API PLATFORM CHALLENGE"

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: SIMULATION NARRATIVE

You are Priya Mehta, the newly appointed API Platform Lead at Meridian Commercial Bank, a mid-size bank with 4 million retail customers, 120,000 SME clients, and USD 28 billion in assets under management. Meridian has received regulatory notification that it must comply with the national Open Banking mandate within 9 months. The bank's existing technology stack consists of a monolithic core banking system (Temenos T24), a basic mobile banking app, and no external API infrastructure.

The CEO has given you three mandates: (1) build a compliant Open Banking API gateway that passes the regulator's conformance tests, (2) create a developer portal that attracts at least 50 registered TPPs within 6 months of launch, and (3) do it without disrupting existing digital banking services. Your budget is USD 2.4 million and your team consists of 6 backend engineers, 2 security specialists, 1 DevOps engineer, and 1 technical writer.

SECTION B2: COMPETING STAKEHOLDER DEMANDS

B2.1 The Compliance Team (Ananya, Chief Compliance Officer)

Ananya's non-negotiable requirements: every API call must be logged for 7 years with full audit trail; no data can leave the bank's sovereign cloud region; all TPP certificates must be validated against the regulator's directory in real-time; consent screens must include mandatory disclosures in the national language; penetration testing must be completed by an independent CREST-certified firm before any production API goes live. Ananya has veto power over launch timelines.

B2.2 The Business Team (Vikram, Head of Digital Banking)

Vikram's priorities are diametrically opposed to Ananya's conservative timeline. He wants Meridian to be the first bank in the market with open APIs to capture early-mover advantage. He is demanding: (1) premium API tiers with monetisation from day one, (2) a 'Powered by Meridian' co-branding requirement for TPPs displaying account data, (3) a data analytics pipeline that gives Meridian insight into which TPP features are most popular with its customers, and (4) an aggressive 4-month launch timeline instead of the regulatory 9-month window.

B2.3 External Developers (Developer Community)

You have conducted pre-launch developer research with 35 fintech companies. Their feedback is clear: (1) they will not integrate with banks that require paper-based onboarding, (2) they need a sandbox with realistic synthetic data (not static mock responses), (3) they want SDKs in Python, Node.js, and Java (not just API docs), (4) they expect sub-200ms API response times, and (5) they will publicly criticise banks with poor DX on developer forums and social media.

SECTION B3: THREE GAME MODES

B3.1 Campaign Mode: The Regulatory Sprint

In Campaign Mode, the intern progresses through a structured 9-month timeline (compressed into 7 project days) with milestone gates. Each milestone requires specific deliverables and unlocks the next phase:

Milestone	Timeline	Deliverable Required	Stakeholder Gate
M1: Foundation	Month 1-2	Gateway architecture document, technology selection rationale, security threat model	Ananya (Compliance) sign-off
M2: Specification	Month 3-4	20 endpoint OpenAPI specs, OAuth/FAPI flow diagrams, consent data model	Technical Architecture Review Board
M3: Developer Portal	Month 5-6	Portal wireframes, documentation structure, sandbox specification, onboarding journey	Vikram (Business) + Developer Advisory Panel
M4: Go-Live	Month 7-9	Rate limiting policy, versioning strategy, monitoring dashboard, incident response playbook	Regulatory Conformance Test + Pen Test

B3.2 Arena Mode: The Developer Experience Audit

In Arena Mode, the intern's developer portal and API documentation are evaluated against a DX benchmark scorecard. The audit simulates a real TPP developer attempting to integrate with Meridian's APIs for the first time. Scoring criteria:

- **Time to First API Call (TTFAC):** Can a developer go from zero (no account) to a successful GET /accounts call in under 30 minutes using only the portal documentation? Score: 0-50 points.

- **Error Recovery:** When the developer encounters a 401 or 403 error, does the error response body provide enough information to self-diagnose and fix? Score: 0-30 points.
- **Sandbox Completeness:** Does the sandbox support the full consent journey, or does it return static mocks? Can the developer trigger error scenarios? Score: 0-40 points.
- **Documentation Accuracy:** Do the code examples actually work when copied and run? Are the request/response examples valid JSON matching the OpenAPI schemas? Score: 0-30 points.

B3.3 Sandbox Mode: The Regulatory Sandbox Review

In Sandbox Mode, the intern's technical artefacts are evaluated against regulatory conformance criteria. This simulates the regulator's technical review before granting production certification. The review checks:

- OpenAPI specifications conform to the published standard (correct endpoint paths, HTTP methods, response codes, and header requirements).
- OAuth/FAPI flows implement all mandatory security controls (PKCE, mTLS, signed request objects, certificate-bound tokens).
- Consent lifecycle is fully specified (creation, authorisation, usage, revocation, expiry) with no gaps.
- Rate limiting policy does not constitute an 'obstacle' to TPP access under PSD2 Article 32(3).
- Sandbox environment meets minimum functional requirements (synthetic data, simulated consent, error injection).
- API versioning and deprecation policy provides adequate migration timelines (minimum 6 months).

SECTION B4: SCORING & PROGRESSION SYSTEM

The simulation tracks intern performance across six dimensions, each contributing to the overall score:

Dimension	Max Points	Assessment Method
Technical Architecture	200	Gateway design document quality, component diagram completeness, technology rationale depth
Regulatory Compliance	200	Consent model accuracy, SCA implementation, audit logging design, certificate validation
Security Engineering	150	OAuth/FAPI flow correctness, threat model coverage, mTLS implementation, token binding
Developer Experience	150	Portal wireframe quality, documentation structure, sandbox specification, onboarding flow
API Governance	150	Rate limiting strategy, versioning policy, deprecation lifecycle, monitoring design
Documentation Quality	150	OpenAPI spec accuracy, README completeness, architecture diagrams, commit message quality

ACHIEVEMENT SYSTEM:

Compliance Champion: All consent lifecycle states correctly specified with no gaps.

DX Pioneer: Developer portal wireframes include interactive 'Try It' API explorer.

Security Sentinel: FAPI 1.0 Advanced profile fully implemented with PAR and JARM.

Specification Maestro: All 20 endpoints pass automated OpenAPI linting with zero warnings.

Sandbox Architect: Sandbox includes error injection, configurable latency, and data reset capability.

SECTION B5: RISK EVENTS & DECISION POINTS

Throughout the simulation, the intern encounters decision points that test their ability to balance competing priorities. These decisions have cascading consequences:

Decision Point 1: Technology Lock-In vs. Speed

Vikram (Business) has found a managed API gateway vendor offering a 90-day free trial with full PSD2 compliance built in. The vendor will have Meridian's open banking APIs live within 6 weeks. However: (1) the vendor uses a proprietary configuration language (no portability), (2) the pricing model escalates sharply after 100,000 monthly API calls, (3) the vendor's sandbox is static (hardcoded responses), and (4) migrating away requires re-implementing all gateway logic from scratch.

Option A - Accept vendor: Go live in 6 weeks, accept lock-in risk. Vikram is delighted, Ananya is nervous about due diligence timelines, developers will struggle with the static sandbox.

Option B - Build on open source: Use Kong or Tyk, go live in 4-5 months. Full control over configuration, no lock-in, dynamic sandbox possible. Vikram is frustrated by the timeline, Ananya approves the architecture review, developers get a better sandbox.

Option C - Hybrid: Use the vendor for production gateway (speed) but build a custom sandbox and developer portal independently. Go live in 8 weeks, moderate lock-in, good DX. Higher total cost but balanced stakeholder satisfaction.

Assessment: The intern must justify their choice with a decision matrix weighing: regulatory risk, vendor lock-in cost, time-to-market value, developer experience impact, and total cost of ownership over 3 years. There is no single correct answer - the quality of the reasoning is assessed.

Decision Point 2: Security Maximalism vs. Usability

Ananya (Compliance) demands that the consent flow require the customer to re-enter their full banking password plus a hardware token OTP for every consent authorisation. She also wants all API responses encrypted at the application layer (in addition to TLS), and all TPP certificates to be manually reviewed by the compliance team before activation (2–3 business day approval). The developer community will revolt: re-entering passwords is poor UX, application-layer encryption adds latency and complexity, and manual certificate review kills self-service onboarding.

Option A - Accept all demands: Maximum security, minimum usability. Expect TPP integration times measured in weeks, not hours. Regulatory conformance is assured but developer adoption will suffer.

Option B - Negotiate smart defaults: Implement SCA with biometric (fingerprint/face) as the primary factor (better UX than passwords), rely on TLS 1.3 for transport security (application-layer encryption is unnecessary if TLS is correctly configured), and implement automated certificate

validation against the regulatory directory (instant approval for trusted certificates, manual review only for edge cases).

Option C - Tiered approach: Basic security for sandbox (self-signed certs, simplified SCA), enhanced security for production (full SCA, directory-validated certificates). Application-layer encryption offered as an optional feature for TPPs that request it.

Decision Point 3: The Hostile TPP

Two months after launch, Meridian's monitoring detects that a registered TPP (FinQuick Ltd) is making 50,000 API calls per hour to the /accounts/{accountId}/transactions endpoint for a single customer, well above normal usage patterns. The TPP appears to be scraping the customer's entire transaction history and caching it locally. This raises concerns: (1) the consent grants 'ReadTransactionsDetail' but does the TPP really need to re-fetch the same transactions every 2 minutes? (2) Is the TPP storing data beyond the consent's data retention period? (3) The excessive API calls are degrading backend performance for other TPPs.

The intern must design the response protocol: rate limiting escalation (warning → throttle → block), communication with the TPP (formal notice citing the terms of service), evidence gathering for potential regulatory escalation, and backend protection (dedicated rate limit bucket for the offending TPP to prevent collateral damage).

SECTION B6: SIMULATION DELIVERABLE MAPPING

Each simulation mode maps to specific project deliverables:

Game Mode	Mapped Deliverable	Assessment Dimension
Campaign M1: Foundation	Gateway architecture doc, technology rationale	API Platform Engineering (200 pts)
Campaign M2: Specification	OpenAPI specs (20 endpoints), consent data model	Open Banking Standards (150 pts)
Campaign M2: Specification	OAuth/FAPI flow diagrams, threat model	OAuth/FAPI Security (200 pts)
Campaign M3: Developer Portal	Wireframes, Getting Started, sandbox spec	Developer Experience (150 pts)
Campaign M4: Go-Live	Rate limiting, versioning, deprecation, monitoring	API Governance (150 pts)
Arena: DX Audit	Documentation quality, code samples, error taxonomy	Documentation Quality (150 pts)
Sandbox: Regulatory Review	Cross-cutting compliance verification	All dimensions (bonus multiplier)

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

CASE STUDY 1: UK OPEN BANKING - MONZO & STARLING'S API-FIRST STRATEGY

Background

Monzo and Starling Bank, UK-based digital challenger banks, adopted an API-first architecture from inception, treating their public APIs not as a regulatory obligation but as a core product. Unlike incumbent banks that bolted API layers onto legacy systems, Monzo built its entire banking platform as a collection of microservices communicating via gRPC internally and exposing RESTful APIs externally. This architectural decision gave them a natural advantage when CMA's Open Banking mandate arrived.

Technical Implementation

Monzo's Open Banking implementation leveraged its existing API gateway (built on a custom Go-based platform) with a dedicated Open Banking adapter layer. Key design decisions: (1) the consent service was implemented as an independent microservice with its own datastore, decoupled from the core authentication system; (2) the sandbox environment was generated from production API schemas with automated synthetic data seeding; (3) the developer portal was built using Stoplight with auto-generated documentation from OpenAPI specs; (4) mTLS was enforced at the gateway level using Open Banking Directory certificates.

Lessons for Interns

- An API-first bank does not simply mean 'we have APIs' - it means the API is the product, not a wrapper around a product.
- Consent must be a first-class microservice, not a flag on a user record. Consent has its own lifecycle, audit requirements, and scaling characteristics.
- Sandbox quality directly correlates with TPP integration speed. Monzo reported that TPPs who used their sandbox integrated 3x faster than those who relied on documentation alone.
- Developer portal is a marketing channel. Monzo's developer documentation became a talent acquisition tool - engineers applied to work at Monzo after reading their API docs.

CASE STUDY 2: INDIA ACCOUNT AGGREGATOR - SETU & SAHAMATI ECOSYSTEM

Background

India's Account Aggregator (AA) framework, operationalised in September 2021, represents a fundamentally different approach to open banking. Rather than mandating banks to expose APIs directly to TPPs, the AA framework introduces a consent-mediated intermediary (the Account Aggregator) that facilitates data flow between Financial Information Providers (banks, insurers, mutual funds, tax authorities) and Financial Information Users (lenders, wealth managers, insurance underwriters).

Technical Implementation

Setu, one of the first technology providers in the AA ecosystem, built its infrastructure on three pillars: (1) a consent management system implementing the Sahamati consent artefact specification (digitally signed JSON with granular purpose, data categories, frequency, and data life parameters); (2) an encrypted data transfer pipeline where financial data is encrypted end-to-end between FIP and FIU - the AA itself never decrypts the data (the 'blind pipe' model); (3) a developer-friendly SDK that abstracted the complex consent flow into a simple widget embeddable in any fintech application.

Lessons for Interns

- The consent artefact model (purpose-bound, time-limited, category-specific) is more privacy-preserving than PSD2's binary consent. Interns should consider how to implement granular consent in their API design.
- The 'blind pipe' architecture ensures that no single entity (including the infrastructure provider) has access to the full data. This has profound implications for data governance and breach containment.
- India's AA framework covers all financial data (bank accounts, mutual funds, insurance, pensions, tax) - far broader than PSD2's payment account scope. Cross-sector data portability is the future direction globally.

CASE STUDY 3: EU PSD2 - TINK'S AGGREGATION PLATFORM

Background

Tink (acquired by Visa in 2022 for EUR 1.8 billion) built a platform that aggregated Open Banking APIs from over 3,400 banks across Europe, providing a unified API abstraction layer for fintech developers. Instead of integrating individually with hundreds of banks (each with slightly different API implementations despite the PSD2 standard), developers integrated once with Tink and accessed all banks through a normalised API.

Technical Challenges

Despite PSD2's standardisation intent, Tink encountered massive implementation inconsistencies across European banks: (1) different API standards (Berlin Group NextGenPSD2 vs. STET vs. UK OBIE vs. Polish API) required separate connector implementations; (2) banks interpreted consent scope differently - some returned 90 days of transactions, others returned only 30; (3) SCA implementation varied wildly (some banks used redirect, others used decoupled OTP, others used embedded SCA); (4) sandbox quality ranged from excellent (UK CMA9 banks) to non-functional (many smaller EU banks).

Lessons for Interns

- Standards are necessary but not sufficient. Even with PSD2, implementation variance across banks created a fragmented ecosystem. The quality of your OpenAPI specification and sandbox determines whether aggregators can integrate efficiently.
- The aggregator business model (Tink, Plaid, Yodlee, TrueLayer) exists precisely because bank-level API integration is too costly for individual fintechs. Your developer portal's quality determines whether TPPs integrate directly or go through an aggregator (reducing your data on TPP behaviour).
- Tink's EUR 1.8 billion acquisition validates that API infrastructure for finance is a multi-billion dollar market. The intern's work on this project represents skills that are commercially valuable at the highest tier.

CASE STUDY 4: AUSTRALIA CDR - FAPI 1.0 ADVANCED IN PRODUCTION

Background

Australia's Consumer Data Right (CDR) is the first open banking regime to mandate FAPI 1.0 Advanced as the security profile for all data sharing. This makes CDR the most technically demanding open banking implementation globally. The CDR also extends beyond banking into energy (electricity, gas) and telecommunications, making it a cross-sector data portability framework.

Technical Implementation

CDR's technical standards (published by the Data Standards Body) mandate: (1) FAPI 1.0 Advanced with PAR (Pushed Authorization Requests) and JARM (JWT Authorization Response Mode); (2) client authentication via private_key_jwt or mTLS; (3) holder-of-key bound access tokens (certificate-bound or DPoP); (4) encrypted ID tokens; (5) a centralised Register with real-time TPP status checking; (6) tiered accreditation with different security requirements per tier.

Lessons for Interns

- FAPI 1.0 Advanced is the direction of travel for all open banking markets. The UK is migrating from its current FAPI-like profile to full FAPI 1.0 Advanced. India's AA v2.0 is aligning with FAPI principles. Building FAPI into your architecture now future-proofs the design.
- CDR's tiered accreditation model (unrestricted, sponsored, representative) provides a useful framework for thinking about developer onboarding tiers in the portal design.
- Cross-sector data portability (banking + energy + telco) means the API gateway architecture must be extensible beyond banking data. The component design should be data-agnostic.

CASE STUDY 5: STRIPE'S DEVELOPER EXPERIENCE - THE GOLD STANDARD

Background

While Stripe is not an open banking provider, its developer portal and API design are universally regarded as the industry benchmark for developer experience. Stripe's documentation has been credited as a primary driver of its adoption - developers choose Stripe not just because of its product but because integrating with Stripe is genuinely pleasant. Every open banking API team should study Stripe's DX practices.

What Stripe Does Differently

- **Interactive API Explorer:** Every endpoint page includes a live API explorer in the right sidebar. Developers can modify request parameters and see real responses against their test-mode account without leaving the documentation page. The explorer auto-populates authentication headers from the developer's dashboard session.
- **Code Samples in 7+ Languages:** Every code example is available in curl, Python, Ruby, PHP, Java, Node.js, Go, and .NET. The language selector is persistent - once a developer selects Python, all code examples across all pages switch to Python.
- **Changelog with Impact Scoring:** Stripe's changelog classifies every change as: 'added' (new feature, non-breaking), 'changed' (modification, may require attention), 'deprecated' (scheduled for removal), or 'removed' (breaking). Each entry links to the affected API version and migration guide.
- **Error Messages That Teach:** Stripe's error responses include not just what went wrong but a direct link to a documentation page explaining how to fix it. The error code 'card_declined' links to a page explaining all possible decline reasons and recommended customer-facing messaging.
- **Webhook Testing Tools:** Stripe provides a CLI tool (stripe listen) that forwards webhook events to a local development server, eliminating the need for ngrok or public URLs during development. This dramatically reduces the friction of testing event-driven integrations.

Lessons for Open Banking Developer Portals

- Time to First API Call is the single most important DX metric. If a developer cannot make a successful API call within 30 minutes of visiting the portal, the onboarding experience has failed.
- Code samples must work when copied. Every code example in the documentation should be tested against the sandbox as part of the CI/CD pipeline. A code example that throws an error when executed is worse than no code example.
- Error messages are documentation. The error response body is the most-read piece of documentation in any API platform. Invest in error taxonomy as a first-class documentation project.

- Developer portal is a conversion funnel. Track: unique visitors → registered developers → sandbox API calls → production API calls. Optimise each step like a product manager optimises a checkout funnel.

CASE STUDY 6: PLAID - AGGREGATOR API DESIGN & CONSENT UX

Background

Plaid connects over 12,000 financial institutions in North America and provides a unified API for fintech applications to access bank account data. Unlike the regulatory-mandated open banking models in Europe and India, Plaid operates in the largely unregulated US market and has historically relied on credential-based access (screen scraping) alongside OAuth-based connections where banks offer them. Plaid's consent UX (Plaid Link) and its API design provide valuable lessons for open banking implementors.

Plaid Link - Consent UX

Plaid Link is an embeddable widget that handles the entire bank connection flow: institution selection, credential entry or OAuth redirect, account selection, and consent confirmation. Key design choices: (1) the widget supports both credential-based and OAuth-based flows, dynamically selecting the right method based on the institution; (2) the consent screen explicitly shows which data categories will be shared (account names, balances, transactions) and for how long; (3) the widget provides real-time status updates during the connection process (connecting, verifying, finalising); (4) error states include actionable recovery suggestions (try a different account, check credentials, institution is experiencing downtime).

Lessons for Interns

- The consent UX is the moment of truth. If the consent screen is confusing, intimidating, or slow, customers will abandon the flow. This translates directly to the TPP's conversion rate and, by extension, the bank's open banking adoption metrics.
- Account selection must be granular. Customers should be able to select which specific accounts to share, not just consent to 'all accounts'. This aligns with data minimisation principles and builds customer trust.
- Real-time connection status reduces abandonment. Showing a progress indicator during the OAuth redirect and token exchange prevents customers from refreshing or navigating away during the critical authentication window.

COMPARATIVE TAKEAWAYS FOR PROJECT DESIGN

Case Study	Key Takeaway for Intern's Design
CS1: Monzo/Starling (UK)	Build consent as an independent microservice with its own datastore. Sandbox quality determines TPP adoption speed.
CS2: Setu/Sahamati (India)	Granular, purpose-bound consent artefacts are more privacy-preserving. The blind-pipe architecture prevents data concentration risk.
CS3: Tink (EU)	Standards alone do not prevent fragmentation. OpenAPI spec quality and sandbox fidelity determine aggregator integration cost.
CS4: CDR (Australia)	FAPI 1.0 Advanced is the global security direction. Design for cross-sector extensibility beyond banking.
CS5: Stripe (DX benchmark)	Time to First API Call < 30 minutes. Error messages are documentation. Code samples must be tested in CI/CD.
CS6: Plaid (Consent UX)	Consent UX is the conversion bottleneck. Granular account selection and real-time status reduce abandonment.

PART D: 7-DAY PROJECT METHODOLOGY & DELIVERABLES

CRITICAL: This is a 7-day AI-accelerated intensive sprint. The intern is expected to use AI-assisted development tools (Claude, GitHub Copilot, ChatGPT) to compress what would traditionally be a 3-6 month architecture engagement into a rigorous 7-day project. AI tools are permitted and encouraged for research, drafting, code generation, and documentation. However, the intern must demonstrate original architectural thinking, critical evaluation of AI outputs, and the ability to identify and correct AI-generated errors.

REPOSITORY STRUCTURE

The intern must create a GitHub repository with the following structure from Day 1:

```
openbanking-api-gateway/
├── README.md
├── SUBMISSION.md
├── .zetheta-project.json
├── docs/
│   ├── architecture/
│   │   ├── gateway-architecture.md
│   │   ├── security-architecture.md
│   │   ├── deployment-topology.md
│   │   └── diagrams/
│   ├── api-specs/
│   │   ├── openapi-accounts.yaml
│   │   ├── openapi-payments.yaml
│   │   ├── openapi-consents.yaml
│   │   └── openapi-events.yaml
│   ├── developer-portal/
│   │   ├── wireframes/
│   │   ├── getting-started.md
│   │   ├── authentication-guide.md
│   │   └── error-reference.md
│   ├── governance/
│   │   ├── rate-limiting-policy.md
│   │   ├── versioning-policy.md
│   │   └── deprecation-policy.md
```

```
│   └── sandbox/
│       ├── sandbox-specification.md
│       └── synthetic-data-model.md
├── src/
│   ├── gateway-config/
│   ├── mock-server/
│   └── postman-collections/
├── tests/
│   ├── api-contract-tests/
│   └── openapi-lint/
└── .github/
    └── workflows/
        └── lint-openapi.yml
```

Day 1 - Regulatory Landscape & Gateway Architecture:

Learning Goals: Understand the regulatory requirements across PSD2, AA, CDR, and UK Open Banking. Design the high-level gateway architecture. Set up the project repository.

- Set up the GitHub repository with the complete folder structure shown above. Initialise with README.md containing the Zetheta metadata header (ZETHETA_INTERN_ID, ZETHETA_PROJECT_CODE: P01D, ZETHETA_PROJECT_TITLE: Open Banking API Gateway & Developer Portal Design).
- Write a regulatory comparison document (docs/architecture/regulatory-comparison.md) comparing PSD2, UK Open Banking, India AA, and Australia CDR across: scope, consent model, security profile, API standard, governance body, and timeline.
- Design the gateway architecture (docs/architecture/gateway-architecture.md) specifying: gateway pattern selection (centralised vs. federated vs. BFF) with rationale, component diagram (TLS termination, authentication engine, request validator, rate limiter, response transformer, audit logger, circuit breaker), technology selection rationale (Kong vs. Apigee vs. AWS API Gateway vs. Tyk vs. custom), deployment topology (cloud region, availability zones, load balancer configuration).
- Create the architecture diagram using Draw.io or Mermaid and commit to docs/architecture/diagrams/.
- Write the .zetheta-project.json metadata file with complete project information.

Deliverable: Committed repository with regulatory comparison, gateway architecture document, component diagram, and project metadata.

Day 2 - OAuth 2.0/FAPI Security Architecture:

Learning Goals: Design the complete OAuth 2.0 and FAPI security architecture including consent management, token lifecycle, and certificate management.

- Write the security architecture document (docs/architecture/security-architecture.md) covering: OAuth 2.0 Authorization Code Flow with PKCE (step-by-step with sequence diagram), FAPI 1.0 Advanced profile implementation (mTLS, signed request objects, PAR, JARM), token lifecycle management (access token, refresh token, ID token - issuance, validation, rotation, revocation), certificate management (eIDAS QWAC/QSEAL for PSD2, OBWAC/OBSEAL for UK, CDR Register certificates for Australia).
- Create detailed sequence diagrams (using Mermaid or Draw.io) for: (a) AIS consent flow (consent creation → authorisation → token exchange → API call), (b) PIS payment initiation flow (payment consent → authorisation → payment submission → status polling), (c) consent revocation flow (customer-initiated → token invalidation → TPP notification).
- Design the consent data model specifying: consent resource schema (JSON), consent states (AwaitingAuthorisation, Authorised, Rejected, Revoked, Expired), consent permissions taxonomy (ReadAccountsBasic, ReadAccountsDetail, ReadBalances, ReadTransactionsBasic, ReadTransactionsDetail, ReadBeneficiaries, ReadStandingOrders, ReadDirectDebits, ReadProducts), and consent-to-token binding.
- Write a threat model document identifying the top 10 security threats to the API gateway (injection, broken authentication, excessive data exposure, rate limiting bypass, consent scope escalation, token theft, certificate spoofing, replay attacks, CSRF in consent flow, insider threat) with mitigation strategies for each.

Deliverable: Security architecture document, three sequence diagrams, consent data model, and threat model. All committed with meaningful commit messages.

Day 3 - OpenAPI 3.0 Specifications (Accounts & Consent):

Learning Goals: Author production-grade OpenAPI 3.0 specifications for the account information and consent management APIs.

- Write the consent API specification (docs/api-specs/openapi-consents.yaml) with endpoints: POST /consents, GET /consents/{consentId}, DELETE /consents/{consentId}. Each endpoint must include: full request/response schemas with $ref components, all mandatory headers (x-fapi-interaction-id, x-fapi-financial-id, Authorization, x-idempotency-key for POST),

complete error response definitions (400, 401, 403, 404, 429, 500), example values for all fields, and security scheme references.

- Write the accounts API specification (docs/api-specs/openapi-accounts.yaml) with endpoints: GET /accounts, GET /accounts/{accountId}, GET /accounts/{accountId}/balances, GET /accounts/{accountId}/transactions, GET /accounts/{accountId}/beneficiaries, GET /accounts/{accountId}/standing-orders, GET /accounts/{accountId}/direct-debits. Include pagination parameters (page, per_page or cursor-based), date range filtering for transactions, and response envelope structure.

- Define all reusable schema components: Account, Balance, Transaction, Consent, Beneficiary, StandingOrder, DirectDebit, ErrorResponse, Links (pagination), Meta (total count, timestamps).

- Validate both specifications using Spectral (OpenAPI linter) with zero errors and zero warnings. Set up a GitHub Actions workflow (.github/workflows/lint-openapi.yml) that runs Spectral on every push.

- Create a Postman collection (src/postman-collections/) with pre-configured requests for all consent and account endpoints, including environment variables for sandbox/production URLs.

Deliverable: Two complete OpenAPI 3.0 YAML files, passing linting, Postman collection, and GitHub Actions workflow.

Day 4 - OpenAPI 3.0 Specifications (Payments & Events):

Learning Goals: Author OpenAPI specifications for payment initiation and event notification APIs. Design the mock server.

- Write the payments API specification (docs/api-specs/openapi-payments.yaml) with endpoints: POST /domestic-payment-consents, GET /domestic-payment-consents/{consentId}, POST /domestic-payments, GET /domestic-payments/{paymentId}, POST /international-payment-consents, GET /international-payment-consents/{consentId}, POST /international-payments, GET /international-payments/{paymentId}. Payment schemas must include: DebtorAccount, CreditorAccount, InstructedAmount (with currency), RemittanceInformation, and Risk assessment fields.

- Write the events API specification (docs/api-specs/openapi-events.yaml) with endpoints: POST /event-subscriptions, GET /event-subscriptions/{subId}, DELETE /event-subscriptions/{subId}, POST /events (polling). Define event types: urn:bank:events:consent-revoked, urn:bank:events:payment-status-changed, urn:bank:events:account-access-consent-linked.

- Build a basic mock server (src/mock-server/) using Prism (Stoplight's mock server) or json-server that serves synthetic responses matching the OpenAPI specifications. The mock server should return different responses based on test account IDs (e.g., account ID starting with 'ERR' returns a 500 error).
- Extend the Postman collection with payment and event endpoints. Create a Postman test suite that validates response schemas against the OpenAPI definitions.

Deliverable: Two additional OpenAPI YAML files (payments and events), mock server configuration, extended Postman collection with test suite.

Day 5: Developer Portal Design & Sandbox Specification:

Learning Goals: Design the developer portal information architecture, create wireframes, and specify the sandbox environment.

- Design the developer portal information architecture (docs/developer-portal/portal-sitemap.md) specifying: navigation structure, page hierarchy, content relationships, and user journey flows for three personas (solo developer, enterprise integration team, compliance officer reviewing TPP documentation).
- Create wireframes for 6+ portal pages using Figma, Miro, or Draw.io: (a) Landing page / homepage, (b) Registration & onboarding flow, (c) Dashboard (after login - showing API keys, usage metrics, sandbox access), (d) API reference page (interactive documentation with 'Try It'), (e) Getting Started guide page, (f) API status page. Export wireframes as PNG and commit to docs/developer-portal/wireframes/.
- Write the Getting Started guide (docs/developer-portal/getting-started.md) that takes a developer from zero to first API call in under 30 minutes. Include code samples in Python, Node.js, and Java for: registering an application, creating a consent, performing the OAuth flow, and making a GET /accounts call.
- Write the authentication guide (docs/developer-portal/authentication-guide.md) with step-by-step OAuth 2.0 / FAPI flow instructions, certificate generation commands, token management best practices, and a troubleshooting FAQ.
- Write the sandbox specification (docs/sandbox/sandbox-specification.md) covering: synthetic data model (how many test accounts, what transaction patterns, what edge cases), simulated consent journey, payment simulation lifecycle, error injection mechanism, rate limit simulation, and data reset capability.

Deliverable: Portal sitemap, 6+ wireframes, Getting Started guide, authentication guide, and sandbox specification.

Day 6 - API Governance, Rate Limiting & Monitoring:

Learning Goals: Design the API governance framework including rate limiting, versioning, deprecation, monitoring, and incident response.

- Write the rate limiting policy (docs/governance/rate-limiting-policy.md) specifying: algorithm selection (token bucket vs. sliding window) with rationale, per-TPP limits by endpoint category (accounts: 300/min, payments: 60/min, bulk: 10/min), global rate limits, burst allowance, rate limit response headers (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset, Retry-After), and escalation procedure for TPPs requesting limit increases.
- Write the versioning policy (docs/governance/versioning-policy.md) specifying: versioning scheme (URI-based /v3.1/), semantic versioning for minor releases, breaking vs. non-breaking change taxonomy, and version lifecycle diagram.
- Write the deprecation policy (docs/governance/deprecation-policy.md) specifying: minimum 6-month deprecation window, Sunset header implementation, migration guide template, dual-running period, sunset enforcement (410 Gone), and communication plan (email, portal changelog, status page).
- Design the monitoring and alerting dashboard (docs/governance/monitoring-design.md) specifying: key metrics (request count, latency P50/P95/P99, error rate by endpoint, consent conversion rate, TPP distribution), alerting thresholds (latency > 1.5s, error rate > 0.5%, availability < 99.5%), and dashboarding tool selection (Grafana, Datadog, or CloudWatch).
- Write the error reference document (docs/developer-portal/error-reference.md) with a complete taxonomy of error codes, each linked to: cause, fix, and example corrected request.

Deliverable: Rate limiting policy, versioning policy, deprecation policy, monitoring design, and error reference document.

Day 7 - Documentation, Testing & GITHUB Transfer:

Learning Goals: Finalise all documentation, run quality checks, and execute the GitHub repository ownership transfer.

- Write the comprehensive README.md with: project overview, architecture diagram (embedded), quick start guide, repository structure, technology stack, standards compliance matrix, API endpoint summary table, and Zetheta attribution.

- Write SUBMISSION.md with: project approach narrative, design decision log (each major decision with rationale and alternatives considered), challenges encountered, AI tools usage disclosure, and self-assessment against the 1000-point rubric.
- Run final quality checks: Spectral OpenAPI linting (zero errors, zero warnings), Markdown lint (markdownlint), link checker (all internal links resolve), image verification (all referenced diagrams exist), and Postman collection validation.
- Write a 10-15 page technical report (docs/technical-report.md) covering: executive summary, regulatory landscape analysis, architecture design rationale, security model analysis, developer experience strategy, governance framework, limitations and future work, and references (20+ citations).
- Execute pre-transfer checklist: all code committed and pushed to main branch, all tests passing, no personal API keys or credentials in the repository, LICENSE file present, .gitignore configured.

GITHUB REPOSITORY OWNERSHIP TRANSFER PROTOCOL

MANDATORY: The completed GitHub repository must be transferred to the ZethetaIntern organisation account. Failure to complete the transfer results in project non-acceptance. No personal forks or copies shall be retained after transfer.

Pre-Transfer Checklist

- All code committed and pushed to main branch (no dangling branches with unmerged work).
- All OpenAPI specs pass Spectral linting in CI/CD pipeline (green checkmark on latest commit).
- Postman collection is exported and committed.
- All wireframes exported as PNG and committed.
- README.md is complete and accurate with Zetheta metadata header.
- No personal API keys, secrets, or credentials in the repository (check .env files, config files).
- LICENSE file present with Zetheta IP attribution.
- .gitignore properly configured (no .DS_Store, no node_modules, no .env).

Transfer Steps

- **Final Commit:** Make a final commit with the message: "Final submission - Open Banking API Gateway v1.0 - [Your Full Name] - [YYYY-MM-DD]".
- **Tag Release:** Create a git tag: git tag -a v1.0 -m "Open Banking API Gateway v1.0 Final Submission".
- **Push Tag:** Push the tag: git push origin v1.0.
- **Transfer Ownership:** Navigate to Settings → Danger Zone → Transfer Ownership. Enter the organisation: ZethetaIntern. Confirm the transfer.
- **Verify Transfer:** Confirm the repository now appears under github.com/ZethetaIntern/openbanking-api-gateway with all commits, branches, and tags intact.
- **Delete Local Copies:** Remove ALL local clones, forks, and copies of the repository from personal devices.
- **Confirmation Email:** Send confirmation email to project supervisor with: (a) transfer timestamp, (b) repository URL, (c) git tag v1.0 hash, (d) Spectral lint summary, (e) brief self-assessment.

Post-Transfer: ZethetaIntern will review the repository within 48 hours. You will receive feedback on architecture quality, specification accuracy, documentation completeness, and governance framework rigour. Outstanding repositories may be selected for patent filing and commercial development.

DAILY SUMMARY MATRIX

This matrix provides a consolidated view of what must be completed each day, the files that must be committed, and the assessment dimensions each deliverable maps to.

Day	Theme	Files to Commit	Primary Dimension	Secondary Dimension
1	Regulatory & Architecture	regulatory-comparison.md, gateway-architecture.md, diagrams/, .zetheta-project.json	API Platform Engineering	Open Banking Standards
2	Security & Consent	security-architecture.md, 3 sequence diagrams, consent-data-model.md, threat-model.md	OAuth/FAPI Security	Open Banking Standards
3	OpenAPI: Accounts & Consent	openapi-consents.yaml, openapi-accounts.yaml, lint-openapi.yml, postman/	Open Banking Standards	Documentation Quality
4	OpenAPI: Payments & Events	openapi-payments.yaml, openapi-events.yaml, mock-server/, postman/	Open Banking Standards	API Platform Engineering
5	Developer Portal & Sandbox	portal-sitemap.md, wireframes/, getting-started.md, authentication-guide.md, sandbox-specification.md	Developer Experience	Documentation Quality
6	Governance & Monitoring	rate-limiting-policy.md, versioning-policy.md, deprecation-policy.md, monitoring-design.md, error-reference.md	API Governance	Documentation Quality
7	Finalise & Transfer	README.md, SUBMISSION.md, technical-report.md, v1.0 tag, repository transfer	Documentation Quality	All dimensions

COMMIT MESSAGE CONVENTIONS

The intern's commit history is assessed as part of the Documentation & Code Quality dimension. Commit messages must follow the Conventional Commits specification:

```
feat: add consent API OpenAPI specification
docs: add OAuth 2.0 sequence diagram for AIS flow
fix: correct international payment consent endpoint path
chore: configure Spectral linting GitHub Action
refactor: restructure payment schemas into reusable components
test: add Postman tests for consent endpoint validation
```

Commit Frequency: A minimum of 3 meaningful commits per day is expected. Single-commit days (entire day's work in one dump) will be penalised under Documentation Quality. The commit history should tell the story of the project's evolution.

Branch Strategy: The intern should work on the main branch for this project. Feature branches are acceptable but must be merged into main before end of day. No orphaned branches should exist at transfer time.

EXPECTED FILE SIZES & QUALITY BENCHMARKS

The following benchmarks indicate the expected depth and thoroughness of each deliverable. These are guidelines, not hard limits - quality matters more than quantity. However, deliverables that fall significantly below these benchmarks are likely to lack sufficient depth.

Deliverable	Expected Length	Quality Indicator
Gateway Architecture (gateway-architecture.md)	1,500-2,500 words	Component diagram with 7+ subsystems, technology selection matrix, deployment topology with AZ/region specification
Security Architecture (security-architecture.md)	2,000-3,000 words	3 sequence diagrams, FAPI 1.0 Advanced controls enumerated, certificate management across 3+ regimes
Threat Model (threat-model.md)	1,500-2,000 words	10+ threats in STRIDE or similar framework, each with likelihood, impact, and specific mitigation
Each OpenAPI YAML file	400-800 lines	All endpoints with full schemas, examples, headers, error responses, security schemes

Deliverable	Expected Length	Quality Indicator
Getting Started Guide	1,000-1,500 words	Code samples in 3+ languages, step-by-step from registration to first API call
Authentication Guide	1,000-2,000 words	OAuth/FAPI flow walkthrough, certificate generation commands, troubleshooting FAQ
Sandbox Specification	1,000-1,500 words	Synthetic data model, simulated consent, error injection, rate limit simulation, data reset
Rate Limiting Policy	800-1,200 words	Algorithm selection, per-endpoint tiers, response headers, regulatory compliance analysis
Versioning Policy	600-800 words	Scheme selection rationale, breaking vs non-breaking taxonomy, version lifecycle
Deprecation Policy	600-800 words	Sunset header, notice period, migration guide template, 410 enforcement
Monitoring Design	800-1,200 words	Metrics catalogue, alerting thresholds, dashboard wireframe or specification
Error Reference	1,000-1,500 words	Complete error code taxonomy with cause, fix, and example corrected request for each
Technical Report	3,000-4,000 words	Executive summary, regulatory analysis, architecture rationale, security analysis, DX strategy, governance framework, limitations, 20+ references
README.md	500-1,000 words	Project overview, embedded architecture diagram, quickstart, repo structure, standards matrix
SUBMISSION.md	800-1,200 words	Approach narrative, decision log, challenges, AI usage disclosure, self-assessment

SAMPLE .zetheta-project.json

The following is the exact structure expected for the project metadata file:

```
{
  "intern_id": "[YOUR_INTERN_ID]",
  "intern_name": "[YOUR_FULL_NAME]",
  "intern_email": "[YOUR_EMAIL]",
  "project_code": "P01D",
   "project_title": "Open Banking API Gateway & Developer Portal
Design",
  "submission_type": "github",
  "submission_date": "[YYYY-MM-DDTHH:MM:SSZ]",
  "tech_stack": [
    "OpenAPI 3.0", "Swagger Editor", "Postman",
    "Spectral", "Draw.io", "Figma", "GitHub Actions"
  ],
  "duration_weeks": 1,
                                                    "github_repo_url":
"https://github.com/ZethetaIntern/openbanking-api-gateway",
  "assessment_criteria": {
    "api_platform_engineering": true,
    "open_banking_standards": true,
    "oauth_fapi_security": true,
    "developer_experience": true,
    "api_governance": true,
    "documentation_quality": true
  }
}
```

SAMPLE OPENAPI 3.0 STRUCTURE (REFERENCE)

The following shows the expected top-level structure of an OpenAPI specification file for this project. This is a structural reference - the intern must write the complete specification with all schemas, examples, and error responses.

```
openapi: 3.0.3
info:
  title: Meridian Bank Open Banking - Accounts API
  version: 3.1.0
  description: >
    Account Information API for Open Banking compliant
    with PSD2/UK OBIE/India AA standards.
  contact:
    name: Meridian API Platform Team
    email: api-support@meridianbank.example
  license:
    name: Proprietary
servers:
  - url: https://sandbox.api.meridianbank.example/v3.1
    description: Sandbox environment
  - url: https://api.meridianbank.example/v3.1
    description: Production environment
security:
  - OAuth2:
      - accounts
paths:
  /accounts:
    get:
      operationId: getAccounts
      summary: List consented accounts
      tags: [Accounts]
      parameters:
        - $ref: '#/components/parameters/x-fapi-interaction-id'
        - $ref: '#/components/parameters/x-fapi-financial-id'
        - $ref: '#/components/parameters/Authorization'
      responses:
        '200':
          description: Successful response
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/AccountsResponse'
        '401':
          $ref: '#/components/responses/Unauthorized'
```

```
        '403':
          $ref: '#/components/responses/Forbidden'
        '429':
          $ref: '#/components/responses/TooManyRequests'
components:
  schemas:
    Account:
      type: object
      required: [AccountId, Currency, AccountType]
      properties:
        AccountId:
          type: string
          format: uuid
          example: 22289-01
        Currency:
          type: string
          pattern: '^[A-Z]{3}$'
          example: GBP
        AccountType:
          type: string
          enum: [Business, Personal]
        AccountSubType:
          type: string
          enum: [CurrentAccount, Savings, CreditCard, Loan]
  securitySchemes:
    OAuth2:
      type: oauth2
      flows:
        authorizationCode:
          authorizationUrl: /authorize
          tokenUrl: /token
          scopes:
            accounts: Access account information
            payments: Initiate payments
```

Note: This reference shows approximately 20% of the expected specification depth. The intern's actual specification must include: all 20 endpoints, complete request/response schemas with nested objects, all mandatory FAPI headers, example values for every field, pagination parameters, date range filters, and comprehensive error response definitions.

SAMPLE SEQUENCE DIAGRAM (MERMAID SYNTAX)

The following shows the expected format for OAuth/FAPI sequence diagrams using Mermaid syntax. The intern must produce at least three diagrams: AIS consent flow, PIS payment flow, and consent revocation flow.

```
sequenceDiagram
    participant Customer
    participant TPP as TPP Application
    participant AuthServer as Bank Auth Server
    participant Gateway as API Gateway
    participant ConsentSvc as Consent Service
    participant AccountSvc as Account Service

    Note over TPP,ConsentSvc: Phase 1 - Consent Creation
    TPP->>Gateway: POST /consents (permissions, expiry)
    Gateway->>ConsentSvc: Create consent resource
    ConsentSvc-->>Gateway: ConsentId, Status=AwaitingAuth
    Gateway-->>TPP: 201 Created (ConsentId)

    Note over Customer,AuthServer: Phase 2 - Authorization
    TPP->>AuthServer: GET /authorize (client_id, consent_id,
                      code_challenge, request_object)
    AuthServer->>Customer: Display login + consent screen
    Customer->>AuthServer: Authenticate (SCA) + Approve
    AuthServer->>ConsentSvc: Update status=Authorised
    AuthServer-->>TPP: 302 Redirect (code, state)

    Note over TPP,AuthServer: Phase 3 - Token Exchange
    TPP->>AuthServer: POST /token (code, code_verifier,
                      client_cert via mTLS)
    AuthServer-->>TPP: access_token, refresh_token, id_token

    Note over TPP,AccountSvc: Phase 4 - API Access
    TPP->>Gateway: GET /accounts (Bearer token, FAPI headers)
    Gateway->>Gateway: Validate token, check consent scope
    Gateway->>AccountSvc: Fetch account data
```

```
    AccountSvc-->>Gateway: Account list
    Gateway-->>TPP: 200 OK (accounts, pagination links)
```

The sequence diagram must be committed as a .mmd (Mermaid) file and also rendered as a PNG image in the diagrams/ directory. Both the source and rendered versions must be present in the repository.

Submission Instructions:

- All files to be named using convention: ProjectCode_YourName_DeliverableName (e.g., 201169C_RahulShah_MainReport)
- All PDFs must be readable and not password-protected
- Supporting Excel files must be in .xlsx format
- All daily deliverables across Days 1–15 are to be submitted on **Day 15** as a consolidated final submission. Students should combine their work into **one or two PDFs and/or one Excel file** wherever possible, rather than submitting separate files for each day. Clearly label each section within the consolidated document by its respective day and topic for ease of assessment.
- Late submissions attract the penalties specified in Part F Assessment Criteria

PART E: TOOLS & PLATFORMS GUIDE

This section provides guidance on the recommended tools for executing the project. All tools listed are either free, open-source, or have free tiers sufficient for this project.

SECTION E1: API DESIGN & SPECIFICATION

Tool	Purpose	Free Tier	Key Feature for This Project
Swagger Editor	OpenAPI 3.0 authoring	Fully free (open source)	Real-time YAML validation, preview, and code generation
Stoplight Studio	Visual API design	Free for individuals	Visual schema builder, mock server, linting rules
Spectral	OpenAPI linting	Fully free (open source)	Custom rulesets, CI/CD integration, severity levels
Postman	API testing & collections	Free (up to 25 requests/run)	Collection runner, environment variables, test scripts
Prism	Mock server from OpenAPI	Fully free (open source)	Dynamic mocking, validation proxy, request logging

SECTION E2: DIAGRAMMING & WIREFRAMING

Tool	Purpose	Free Tier	Key Feature for This Project
Draw.io (diagrams.net)	Architecture diagrams	Fully free	AWS/Azure/GCP icons, sequence diagrams, flowcharts
Miro	Collaborative whiteboarding	Free (3 boards)	Wireframing, user journey mapping, sticky notes
Figma	UI wireframing	Free (3 files)	Developer portal wireframes, component libraries
Mermaid	Code-based diagrams	Fully free (open source)	Sequence diagrams, flowcharts, embedded in Markdown
PlantUML	UML diagrams	Fully free (open source)	Sequence diagrams, class diagrams, state machines

SECTION E3: DOCUMENTATION & COLLABORATION

Tool	Purpose	Free Tier	Key Feature for This Project
Notion	Documentation hub	Free for personal use	Markdown, databases, embedded diagrams, sharing
ReadMe	Developer portal simulation	Free (1 project)	API reference from OpenAPI, changelog, Getting Started
GitHub Pages	Static site hosting	Fully free	Host API docs, portal prototype from the repository
MkDocs	Documentation site generator	Fully free (open source)	Markdown to HTML, search, versioning

SECTION E4: AI-ASSISTED DEVELOPMENT

AI tools are permitted and encouraged for this project. Proper attribution and critical evaluation of AI outputs is required.

Tool	Permitted Use	Attribution Required
Claude (Anthropic)	Research, drafting, code generation, review	Yes - document which sections were AI-assisted in SUBMISSION.md
GitHub Copilot	Code completion, YAML generation	Yes - note in commit messages if Copilot-assisted
ChatGPT (OpenAI)	Research, alternative perspectives	Yes - document in SUBMISSION.md
Cursor / Windsurf	Code editing, refactoring	Yes - note in SUBMISSION.md

AI INTEGRITY POLICY: While AI tools may generate initial drafts, the intern is responsible for: (1) verifying all technical claims and regulatory references, (2) identifying and correcting AI hallucinations, (3) adding original architectural reasoning that goes beyond AI-generated boilerplate, (4) demonstrating critical thinking through design decision rationale. Submissions that are unmodified AI outputs with no evidence of critical evaluation will receive significant penalties under the Documentation Quality dimension.

SECTION E5: ESSENTIAL READING LIST

The following resources are essential pre-reading for this project. Interns are expected to reference these materials in their technical report.

Regulatory & Standards Documents

- **PSD2 Regulatory Technical Standards (RTS) on SCA & CSC:** European Banking Authority Delegated Regulation (EU) 2018/389. The foundational document defining Strong Customer Authentication requirements and common and secure communication standards.
- **UK Open Banking API Specifications v3.1:** Published by the Open Banking Implementation Entity (OBIE). Defines the Read/Write Data API, Payment Initiation API, Event Notification API, and Dynamic Client Registration API with full OpenAPI 3.0 specifications.
- **India Account Aggregator API Specification v2.0:** Published by ReBIT (Reserve Bank Information Technology). Defines the consent flow, data flow, and notification APIs for the AA ecosystem.
- **Australia CDS API Standards:** Published by the Data Standards Body (consumerdatastandards.gov.au). The most technically demanding open banking standard, mandating FAPI 1.0 Advanced.
- **FAPI 1.0 Advanced Security Profile:** OpenID Foundation specification. Defines the security hardening of OAuth 2.0 for financial APIs including mTLS, PAR, JARM, and signed request objects.
- **Berlin Group NextGenPSD2 Framework v1.3:** The most widely adopted PSD2 API standard across continental Europe. Defines Account Information, Payment Initiation, and Confirmation of Funds APIs.

Technical References

- **RFC 6749 - The OAuth 2.0 Authorization Framework:** The foundational OAuth 2.0 specification. Required reading for understanding authorization flows.
- **RFC 7636 - PKCE (Proof Key for Code Exchange):** Extension to OAuth 2.0 preventing authorization code interception. Mandatory for all open banking implementations.
- **RFC 8705 - OAuth 2.0 Mutual-TLS Client Authentication:** Defines mTLS-based client authentication and certificate-bound access tokens for FAPI compliance.
- **RFC 9126 - Pushed Authorization Requests (PAR):** Defines the PAR endpoint for pushing authorization requests via backchannel. Required for FAPI 1.0 Advanced.

- **RFC 8594 - The Sunset HTTP Header Field:** Defines the Sunset header for communicating API deprecation dates in a machine-readable format.
- **OpenAPI Specification 3.0.3:** The technical standard for describing RESTful APIs. Required for all API specification deliverables.

Industry Analysis & DX Best Practices

- **Stripe API Documentation:** stripe.com/docs - The benchmark for developer experience design. Study the information architecture, code sample presentation, and error handling documentation.
- **Plaid Developer Documentation:** plaid.com/docs - Example of aggregator API design with embeddable consent UX (Plaid Link).
- **TrueLayer Developer Portal:** truelayer.com/docs - European open banking aggregator with strong DX focus. Study the Getting Started guide and authentication walkthrough.
- **Open Banking UK Directory & Trust Framework:** directory.openbanking.org.uk - The operational trust framework for UK open banking. Study the TPP registration and certificate issuance process.
- **Sahamati Account Aggregator Dashboard:** sahamati.org.in - Live statistics and ecosystem participants for India's AA framework.

SECTION E6: GITHUB WORKFLOW TEMPLATES

The following CI/CD workflow templates should be adapted for the project repository:

OpenAPI Linting Workflow (GitHub Actions)

```
name: Lint OpenAPI Specifications
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm install -g @stoplight/spectral-cli
```

```
      - run: spectral lint docs/api-specs/*.yaml --fail-severity
warn
```

Markdown Linting Workflow

```
name: Lint Markdown
on: [push, pull_request]
jobs:
  markdown-lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DavidAnson/markdownlint-cli2-action@v16
        with: { globs: '**/*.md' }
```

Link Checker Workflow

```
name: Check Links
on: [push]
jobs:
  links:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: lycheeverse/lychee-action@v1
        with: { args: '--verbose docs/' }
```

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

END OF PROJECT DOCUMENT